

- Legal QA RAG — Technical Documentation
 - Table of Contents
 - 1. System Overview
 - 2. Technology Stack
 - 3. Repository Layout
 - 4. Document Corpus
 - 5. Configuration
 - ChunkingConfig (config.py:27)
 - EmbeddingConfig (config.py:34)
 - RetrievalConfig (config.py:42)
 - StorageConfig + PipelineConfig (config.py:53)
 - 6. Data Models
 - SourceDocument (models.py:8)
 - DocumentChunk (models.py:18)
 - RetrievedChunk (models.py:31)
 - 7. Ingestion Pipeline
 - 8. Text Normalization & PDF Repair
 - 8.1 Load-time normalization — loaders.py
 - 8.2 fix_pdf_spacing — text_normalization.py:172
 - 8.3 BM25 tokenizer — tokenize_for_search (text_normalization.py:198)
 - 9. Heading Detection & Document Hierarchy
 - 10. Chunking
 - 10.1 Pre-processing
 - 10.2 Heading-driven chunk emission (chunking.py:390)
 - 10.3 Recursive splitting (_recursive_split, chunking.py:204)
 - 11. Document Catalog
 - 11.1 Ingest-time helpers (used by loaders.py)
 - 11.2 DocumentCatalog.from_chunks (:225)
 - 11.3 Query matching — match_query(query) (:400)
 - 12. Embeddings
 - 13. Vector Storage (Qdrant)
 - 14. Hybrid Retrieval
 - 14.1 Channels
 - 14.2 Fusion — weighted Reciprocal Rank Fusion (_fuse, :109)
 - 14.3 search(query) orchestration (:255) — full flow
 - 15. Query Understanding

- 15.1 Section number extraction — `_extract_section_from_query` (retrieval.py:570)
- 15.2 Document context — `_extract_document_context` (:366)
- 16. Ranking: Filters & Boosts
 - 16.1 Named-instrument filter (`_filter_candidates_by_named_instrument`, :174)
 - 16.2 Document boosts
 - 16.3 Section keyword filter (`_filter_by_section_keyword`, :652)
 - 16.4 Section boost (`_boost_section_matches`, :708)
 - 16.5 Parent-child boost (`_boost_parent_child_matches`, :481)
 - 16.6 Amendment-aware boost (`_boost_amendment_matches`, :513)
 - 16.7 Multi-document coverage (`_ensure_multi_document_coverage`, :421)
- 17. Intent Classification & Prompt Engineering
 - 17.1 `classify_query_intent` (prompts.py:27)
 - 17.2 System prompt (`SYSTEM_PROMPT`, prompts.py:134)
 - 17.3 Intent templates (prompts.py:175–416)
 - 17.4 Context assembly — `build_legal_prompt` (prompts.py:545)
- 18. LLM Layer & Adaptive Reasoning
- 19. API Server & SSE Streaming Protocol
 - Endpoints
 - `/chat` flow (api.py:140)
 - SSE event schema (each line data: `<json>\n\n`, `_sse`, api.py:78)
- 20. Frontend
 - `App.jsx`
 - `utils/api.js` — SSE client
 - `ChatArea.jsx`
 - `MessageBubble.jsx`
 - `Sidebar.jsx`
- 21. Shared Utilities (rag_pipeline/utils.py)
- 22. Operations
- 23. Known Gotchas & Fixed Bugs
- 24. Limitations & Future Work

Legal QA RAG — Technical Documentation

Complete technical reference for the Pakistani-law question-answering system: every component, data structure, algorithm, and design decision, with file/line pointers.

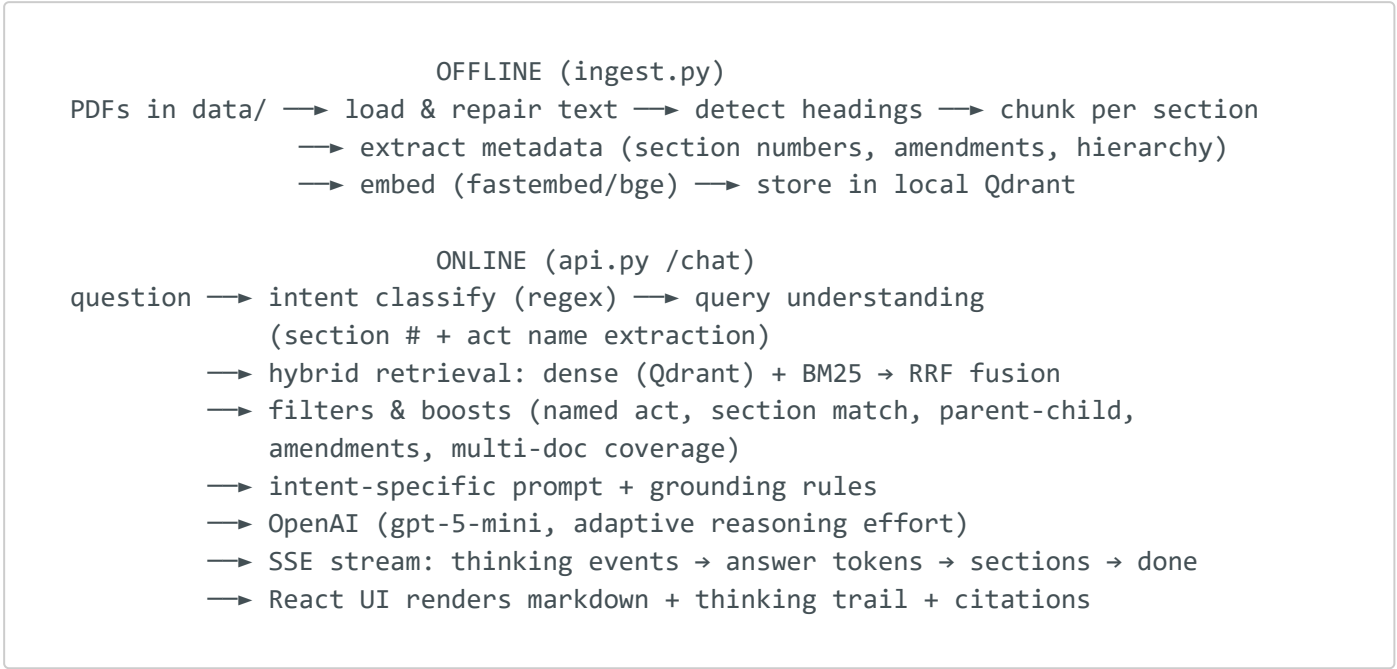
Companion docs: [REASONING_MECHANISM.md](#) (adaptive LLM reasoning), [RESEARCH_PAPER.md](#) (academic write-up), [README.md](#) (quick start).

Table of Contents

1. [System Overview](#)
 2. [Technology Stack](#)
 3. [Repository Layout](#)
 4. [Document Corpus](#)
 5. [Configuration](#)
 6. [Data Models](#)
 7. [Ingestion Pipeline](#)
 8. [Text Normalization & PDF Repair](#)
 9. [Heading Detection & Document Hierarchy](#)
 10. [Chunking](#)
 11. [Document Catalog \(Act Name / Abbreviation Resolution\)](#)
 12. [Embeddings](#)
 13. [Vector Storage \(Qdrant\)](#)
 14. [Hybrid Retrieval](#)
 15. [Query Understanding](#)
 16. [Ranking: Filters & Boosts](#)
 17. [Intent Classification & Prompt Engineering](#)
 18. [LLM Layer & Adaptive Reasoning](#)
 19. [API Server & SSE Streaming Protocol](#)
 20. [Frontend](#)
 21. [Shared Utilities](#)
 22. [Operations: Running, Indexing, Debugging](#)
 23. [Known Gotchas & Fixed Bugs](#)
 24. [Limitations & Future Work](#)
-

1. System Overview

A local Retrieval-Augmented Generation (RAG) system that answers questions about Pakistani statutes (drug control, criminal procedure, evidence law, penal code, police rules, anti-money-laundering, etc.). The pipeline:



Key design principles:

- **No hardcoded act lists.** Everything about the documents (titles, aliases, acronyms, groups) is derived at ingest/index time from filenames, content, and optional `.meta.json` sidecars (`document_catalog.py:1`).
- **Grounding over fluency.** The prompts contain explicit anti-hallucination rules (never relabel sections, refuse when not found) because real failures occurred (see §23).
- **Lexical + semantic.** Legal lookups are keyword-dominant ("Section 9 CNSA"), so BM25 carries 55% of the fusion weight; dense embeddings carry semantics for scenario questions.
- **Cost-scaled reasoning.** Regex intent analysis (free) gates LLM reasoning effort (expensive) — see `REASONING_MECHANISM.md`.

2. Technology Stack

Layer	Technology	Where
PDF extraction	<code>pypdf</code>	<code>rag_pipeline/loaders.py:7</code>

Layer	Technology	Where
Embeddings	<code>fastembed</code> (ONNX, CPU) — <code>BAAI/bge-base-en-v1.5</code> , 768-dim	<code>rag_pipeline/embeddings.py</code>
Vector DB	Qdrant local mode (embedded, file-backed at <code>qdrant_data/</code>)	<code>rag_pipeline/storage.py</code>
Lexical search	<code>rank_bm25</code> (BM25Okapi, in- memory)	<code>rag_pipeline/retrieval.py:10</code>
API	FastAPI + uvicorn, SSE via <code>StreamingResponse</code>	<code>api.py</code>
LLM	OpenAI Chat Completions — default <code>gpt-5-mini</code> (reasoning model)	<code>api.py:35</code>
Frontend	React 18 + Vite, <code>react-</code> <code>markdown</code> + <code>remark-breaks</code> , <code>axios/fetch</code>	<code>frontend/</code>
Config	dataclasses + <code>.env</code> via <code>python-dotenv</code>	<code>rag_pipeline/config.py</code> , <code>api.py:8</code>

Full dependency list: `requirements.txt`. (`pdfplumber`, `pdf2image`, `PyMuPDF`, `reportlab`, `scikit-image` are listed but unused by the current pipeline — only `pypdf` does extraction.)

3. Repository Layout

```

anf/
├─ api.py                # FastAPI server, SSE streaming, LLM calls,
reasoning effort
├─ ingest.py             # CLI for indexing documents (incremental / force)
├─ requirements.txt
├─ .env / .env.example   # OPENAI_API_KEY, OPENAI_MODEL
├─ data/                 # source PDFs (+ optional *.meta.json sidecars)
├─ qdrant_data/          # local Qdrant collection storage
├─ rag_pipeline/
└─ ── _init_.py          # exports PipelineConfig, RAGPipeline

```

```

|   |— config.py           # all tunable dataclasses
|   |— models.py          # SourceDocument, DocumentChunk, RetrievedChunk
|   |— loaders.py         # discovery, PDF/text extraction, normalization
|   |— text_normalization.py # PDF spacing repair, BM25 tokenizer
|   |— hierarchy.py       # heading detection (the parser's heart)
|   |— chunking.py        # section-aware chunking + metadata extraction
|   |— document_catalog.py # act-name/alias/acronym catalog + query matching
|   |— embeddings.py      # fastembed wrapper
|   |— storage.py         # Qdrant local store wrapper
|   |— retrieval.py       # hybrid search, all filters & boosts (the largest
module)
|   |— prompts.py        # intent classifier, system prompt, 6+2 templates
|   |— rag.py            # RAGPipeline orchestrator (ingest + context
building)
|   |— utils.py          # section-number normalization, reflow, citation
builder
|   |— frontend/
|       |— src/
|           |— App.jsx           # conversation state, theming
|           |— components/
|               |— ChatArea.jsx  # input, streaming message updates
|               |— MessageBubble.jsx # markdown render + ThinkingBlock
|               |— Sidebar.jsx   # conversation list, search, theme toggle
|           |— utils/api.js      # SSE client (fetch + reader)
|           |— styles/*.css
|   |— *.log               # server + reindex logs

```

4. Document Corpus

20 PDFs in `data/` (indexed: **10,485 chunks** as of the 2026-06-03 rebuild):

- **Drug law:** Control of Narcotic Substances Act 1997 (+ First Amendment 2020, Amendment Act 2022), Dangerous Drugs Act 1930, Control Substances (Regular Drugs of abuse), ANF Act 1997, National Anti-Narcotics Policy 2019
- **Criminal law:** Pakistan Penal Code, Code of Criminal Procedure 1898, Prevention of Smuggling Act 1977
- **Evidence:** Qanun-e-Shahadat Order 1984
- **Financial crime:** Anti-Money Laundering Act 2010 (amended up to Sep 2020)
- **Service/police law:** Civil Servants Act 1973, Punjab Police Rules vols I–III, Punjab Police Efficiency & Discipline Rules 1975
- **Misc:** Disposal of Vehicles and other Articles, Inter-Agency Task Force

Sidecar metadata (`<file>.pdf.meta.json`, loaded by

`document_catalog.load_sidecar_metadata`, `document_catalog.py:101`) can override `title`, `short_names` (extra aliases), `document_type`, `document_group_id`,

and `amends/amends_group_id`. Sidecars are explicitly excluded from ingestion as content (`loaders.py:36 _is_sidecar`).

5. Configuration

All defaults live in dataclasses in `rag_pipeline/config.py`; `ingest.py` exposes a few as CLI flags. (The `.env.example` retrieval values are *historical* — the code defaults in `config.py` are what actually apply; only `OPENAI_API_KEY` / `OPENAI_MODEL` are read from `.env` at runtime.)

ChunkingConfig (`config.py:27`)

Field	Default	Rationale
<code>max_chars</code>	2000	legal definitions are long
<code>overlap_chars</code>	250	preserve context across forced splits
<code>min_chars</code>	100	some sections are one line ("20B. Confiscation.")

EmbeddingConfig (`config.py:34`)

Field	Default
<code>model_name</code>	BAAI/bge-base-en-v1.5
<code>batch_size</code>	16 (auto-halves on failure, <code>embeddings.py:55</code>)
<code>use_gpu</code>	False (CPU-only ONNX providers)

RetrievalConfig (`config.py:42`)

Field	Default	Meaning
<code>dense_top_k</code>	20	candidates pulled from each of dense and BM25
<code>hybrid_top_k</code>	15	survivors of RRF fusion
<code>rerank_top_k</code>	10	final results returned to the API

Field	Default	Meaning
<code>rrf_k</code>	60	RRF dampening constant
<code>bm25_weight</code>	0.55	keyword matches dominate legal lookups
<code>dense_weight</code>	0.45	
<code>reranker_model</code>	None	cross-encoder hook, not wired up

StorageConfig + PipelineConfig (config.py:53)

- Collection name = `rag_` + sanitized embedding model name (`rag_baai_bge_base_en_v1_5`), overridable via `custom_collection_name` (`config.py:69`).
- `data_dir="data"`, `fallback_dir="."` — if `data/` is empty the pipeline scans the project root (`rag.py:45 discover_sources`).
- `supported_extensions`: pdf/txt/md/csv/json/yaml/log/rst/ini/toml/xml/html (`config.py:7`), though the loader only has extractors for PDF vs plain-text.

6. Data Models

All in `rag_pipeline/models.py` (slotted dataclasses):

SourceDocument (models.py:8)

One loaded file: `document_id` (resolved posix path), `source_path`, `title`, `pages: list[str]` (normalized per page), `text` (pages joined by `\n\n`), `metadata` (`page_count`, `file_name`, `display_title`, `document_type`, `document_group_id`, `search_aliases`, optionally `amends_group_id`).

DocumentChunk (models.py:18)

One indexed unit: `chunk_id` (`<document_id>::<00042>`), `document_id`, `source_path`, `text` (prefixed with `SECTION: <path>` when inside a section),

`section_path`: `tuple[str, ...]` (heading-stack at emit time), `chunk_index`, `start_char/end_char`, and `metadata`:

Key	Source
<code>title</code> , <code>display_title</code> , <code>file_name</code> , <code>page_count</code>	document level (<code>loaders.py:109</code>)
<code>document_type</code> (<code>base_act/amendment/rules/policy/reference/unknown</code>)	folder name → title heuristics (<code>document_catalog.py:123</code>)
<code>document_group_id</code>	subject key, e.g. <code>control-narcotic-substances</code> (<code>document_catalog.py:159</code>)
<code>search_aliases</code>	title + humanized stem + generated acronyms (<code>document_catalog.py:200</code>)
<code>amends_group_id</code>	sidecar or stem heuristic (<code>loaders.py:117</code>)
<code>section_path</code> , <code>section_label</code>	heading stack (<code>chunking.py:346</code>)
<code>section_number</code>	extracted from heading (<code>chunking.py:283</code>)
<code>parent_section</code> , <code>section_hierarchy</code> , <code>subsection_depth</code>	hierarchy metadata (<code>chunking.py:336</code>)
<code>amendments_detected</code> , <code>amendments</code> , <code>amendment_references</code>	regex amendment scan (<code>chunking.py:149</code>)

RetrievedChunk (`models.py:31`)

A chunk + scores: `dense_score` (cosine), `lexical_score` (BM25), `fused_score` (RRF, then multiplied by every boost), `rerank_score` (unused).

7. Ingestion Pipeline

Entry: `python ingest.py [--data-dir data] [--force-reindex] [--model ...] [--embed-batch-size N] [--local-path qdrant_data] [--extensions .pdf,.md] (ingest.py:21)`.

`RAGPipeline.ingest(force_reindex)` (`rag.py:51`) does:

1. **Discover** — `discover_documents` recursively globs `data_dir` (fallback to `fallback_dir` if empty), filters by extension, skips `.meta.json`, dedupes and sorts by resolved path (`loaders.py:42`).
2. **Incremental check** (default) — `store.scroll_all()` collects every stored chunk's `source_path`; files already present are skipped. `--force-reindex` bypasses this and later calls `store.reset()` for a clean rebuild (`rag.py:62`).
3. **Load** — `load_document` per file (`loaders.py:95`): pypdf page-by-page extraction → per-page normalization (§8) → sidecar/title/type/group/alias metadata.
4. **Chunk** — `chunk_document` (§10).
5. **Embed** — `embed_documents` over all chunk texts (§12).
6. **Store** — `reset()` (force) or `ensure_collection_exists()` (incremental), then `upsert` (§13).
7. **Refresh** — `retriever._refresh_indexes()` rebuilds the in-memory BM25 index and the DocumentCatalog from the stored payloads (`retrieval.py:65`).

The API server builds the same `RAGPipeline` at startup (`api.py:99`) — it never ingests, it only loads what's already in `qdrant_data/`.

8. Text Normalization & PDF Repair

PDF extraction of these statutes is messy: collapsed spacing, hyphenated line breaks, footnote markers fused to words, run-on tokens. Two stages clean it.

8.1 Load-time normalization — `loaders.py`

`_normalize_text` (`loaders.py:70`), applied per page:

1. Strip NULs, normalize CRLF → LF.
2. De-hyphenate line-broken words: `-\\n` followed by lowercase is joined (`loaders.py:73`).
3. Trim trailing whitespace per line; collapse 3+ blank lines to one.

4. `_remove_legal_annotations` (`loaders.py:58`):
 - drops `vide amendments ... dated ...` gazette citations;
 - strips inline footnote digits glued to words (`held51` → `held`) — pattern is deliberately `[A-Za-z]` before digits, **not** `\w`, because `\w` includes digits and would corrupt `103`. headings (`loaders.py:63`);
 - removes `[see section ...]` bracketed cross-refs.
5. `fix_pdf_spacing` (below).

8.2 `fix_pdf_spacing` — `text_normalization.py:172`

Repairs collapsed spacing **line by line** (critical: an earlier version used `str.split()` over the whole text, which destroyed every newline and broke heading detection — the "0% section coverage" bug):

1. camelCase split: `theCourt` → `the Court` (`_CAMEL_SPLIT_RE`).
2. Space around `,` `;` `:` punctuation.
3. **Run-on token repair** (`_repair_runon_token, :142`): for alphabetic tokens ≥ 12 chars, `_segment_runon_word (:120)` greedily decomposes the token into known function/legal words (the ~ 110 -word `_COMMON_WORDS` list, longest-first). The split is accepted **only if** the entire token decomposes into dictionary words **and** no segment is shorter than 3 chars — otherwise the token is left untouched. (A previous version fell back to single-character splits and shattered ordinary words: `magistrates` → `m a g i s t r a t e s`.)
4. Collapse runs of spaces; trim around newlines.

8.3 BM25 tokenizer — `tokenize_for_search` (`text_normalization.py:198`)

Applies `fix_pdf_spacing`, lowercases, and additionally splits residual run-on tokens of 20+ letters by case/acronym chunks (`_ACRONYM_CHUNK_RE`) so corrupted corpus tokens still match query terms. Used for both indexing chunks and queries (`retrieval.py:40`).

9. Heading Detection & Document

Hierarchy

`hierarchy.detect_heading(line)` (`hierarchy.py:95`) classifies a single line as a heading (with level) or returns `None`. This single function determines section boundaries and therefore section metadata quality. Order of checks:

Pre-filters (reject/clean before matching):

- **0a.** Strip leading `□" "'` insertion markers — amended provisions are printed bracketed (`[93-A. Sending of summons...]`); without stripping, inserted sections were invisible (`hierarchy.py:104`).
- **0.** Reject **table-of-contents lines** — dotted leaders or trailing page numbers (`_TOC_LINE, :34`). Without this every section existed twice (TOC stub
 - body) producing phantom duplicates.
- **0b.** Reject **page footers/headers** — `3 | Page, 10 | P a g e, Page 12` (`_PAGE_MARKER, :39`); their leading digit otherwise becomes a phantom section number that hijacks lookups.

Pattern hierarchy (first match wins):

#	Pattern	Example	Level
1	<code>_LEGAL_SECTION:</code> <code>Section/Article/Rule/Schedule <num></code> <code>[title]</code>	Article 20B ..., Rule 12.14 ...	1 + dots-in- num
2	<code>_LABELED_HEADING:</code> <code>Chapter/Part/Schedule ...</code>	CHAPTER II	1
3	<code>_NUMERIC_HEADING: <num><delim> Title</code>	161. Police officer..., 337-A. Punishment...	1 + dots-in- num
4	<code>_ALPHA_HEADING: single letter</code>	a) ...	4
5	<code>_ROMAN_HEADING</code>	iv) ...	5
6	<code>_looks_like_title: Title-Case/ALL-CAPS</code> line	PRELIMINARY	1

The shared section-number regex (`_SECTION_NUM_PATTERN, :12`, duplicated as `_SEC_NUM` in `retrieval.py:29` and `chunking.py:293` — they must stay in sync):

```
\d+(?:\.\d+)*(?:[A-Z]{1,3})?(?:[-_][A-Za-z]{1,6})?(?:\([a-z\d]+\))*
```

covers 161, 3.1.4, 20B, 337-I, 45(1)(a), Rule 12.14.

Anti-phantom guards (each fixed a real bug):

- `_is_running_text_after_number (:53)`: a number followed by a lowercase word (optionally after a stray comma) is **wrapped sentence text or a cross-reference, not a heading** — "Section 30 , may pass any sentence...", "339 a person who...". Treating these as headings split real provisions and spawned phantom sections (a wrapped line once stole section 34's entire body). A comma followed by Title-Case survives, so OCR'd 265-N , Place of holding sittings : still detects.
- `_looks_like_title (:74)` rejects lines ending in . ; , - - -. The dash rule is the "Shall---" fix: a wrapped provision emitting Shall--- became a level-1 heading that wiped the real section heading off the stack (CNS Act s.8 lost its number; every following section was mis-parented). Also requires: 4–90 chars, ≤12 words, ≥70% ALL-CAPS or ≥85% Title-Case words.

`section_path_label (:188)` joins the heading stack with " > " for display.

10. Chunking

`chunk_document(document, config)` (`chunking.py:268`) — section-aware chunking with a recursive fallback.

10.1 Pre-processing

1. `_normalize_block + fix_pdf_spacing` once more over the whole document.
2. `_merge_split_section_number_lines` (`chunking.py:28`) — two passes that repair PDF line-splitting of headings:
 - **Dangling suffix**: 337- on one line, A. on the next → rejoined to 337-A. (PPC hurt provisions extract this way).
 - **Number-only line**: 103. alone, title on next line → 103. Exclusion of evidence.... Guarded: next line must be ≥6 chars, not itself a number/heading, and start alphabetically.

10.2 Heading-driven chunk emission (`chunking.py:390`)

The text is walked line by line with a **heading stack** (`section_stack` of `_HeadingState(level, title)`):

- On a heading: flush the buffered body (`emit_buffer`), pop stack entries with `level >= new_level`, push the new heading.
- Otherwise: buffer the line.

`emit_buffer` (`chunking.py:315`) per buffered section body:

1. Compute `section_path` (stack titles) and its label.
2. **Section number extraction** (`_extract_section_number, :283`) — walks the path bottom-up; per segment tries, in order: explicit `Section/Article/Rule X` at start → leading bare number → `Section X` anywhere. The ordering fixes the "13. Punishment for contravention of Section 7" case (own number 13 must beat cross-ref 7).
3. **Amendment detection** (`_detect_amendments, :149`) — regexes for `as amended by...`, `substituted/inserted/omitted...`, `now reads:`; captured section refs go into `amendment_references`.
4. **Hierarchy metadata** — `parent_section` (path root), `section_hierarchy` (full path list), `subsection_depth` (count of `(` in the heading: `45(1)(a)` → 2).
5. `_recursive_split` to respect `max_chars` (below).
6. Each piece becomes a `DocumentChunk` whose text is prefixed `SECTION: <path label>\n\n<body>` so the section identity travels inside the embedded text too. Document-level metadata is merged into each chunk's metadata.

10.3 Recursive splitting (`_recursive_split, chunking.py:204`)

Hierarchical fallback for oversized bodies (>2000 chars): **paragraphs** (`\n\s*\n`) → greedy paragraph packing → oversized paragraphs split by **sentences** (`((?<=[.!?])\s+)`) → oversized sentences split by **words** with a 250-char word-level overlap window (`_split_by_words, :111`).

If the whole document produced no headings, the entire text is recursively split into path-less chunks (`chunking.py:402`).

11. Document Catalog

`document_catalog.py` builds, **from indexed chunks alone**, the knowledge needed to resolve "which Act does the user mean" — including abbreviations and typos. No hardcoded file lists; entries are inert unless a matching document exists.

11.1 Ingest-time helpers (used by `loaders.py`)

- `humanize_stem (:116)`: `anti-narcotics-force-act-1997` → `Anti Narcotics Force Act 1997`.
- `infer_document_type (:123)`: `sidecar` → folder name (`amendments/`, `rules/...`) → title keywords (`amendment`, `rules`, `policy`, `act/code/ordinance`).
- `infer_group_id (:159)`: a stable *subject key* shared by a base Act and its amendments/volumes — strips parentheticals, non-letters, noise words (`_GROUP_NOISE`: `act/amendment/upto/ordinals/months/roman numerals...`); e.g. all three CNS instruments collapse to `control-narcotic-substances`.
- `generate_aliases (:200)`: title + sidecar short_names + humanized stem + **acronym candidates** (`_acronym_candidates`, `:176`): initial letters of significant title words, with and without the instrument-type word — `Control of Narcotic Substances Act 1997` → `{CNS, CNSA}`. Years/roman/noise never contribute letters, so base acts and amendments share acronyms.

11.2 `DocumentCatalog.from_chunks (:225)`

Rebuilt on every `_refresh_indexes`. Groups chunks by `document_id` into `DocumentRecords` (title, aliases, group, type, amends-link, paths); builds a longest-first alias list and a per-doc acronym set (re-derived at build time so old indexes without stored aliases still work).

11.3 Query matching — `match_query(query) (:400)`

Returns the display title of the act the query names, via four prioritized stages:

1. **Verbatim alias substring** (hyphen/space-normalized). Aliases ≤ 8 chars (`anf`, `crpc`) must match as whole words so they can't fire inside other words.
2. **Abbreviation match** (`_match_abbreviation`, `:361`): dots collapsed (`C.N.S.A` $\rightarrow$ `cnsa`); each 3–8-letter query word is checked against (a) the document's generated acronyms, and (b) `_KNOWN_ABBREVIATIONS` (`:62`) — curated short forms whose conventional abbreviation isn't plain initials (`crpc`, `ppc`, `qso`, `amla`, `dda`, ...), each mapped to distinctive title tokens that must ALL belong to the candidate document. Ties go to the **base act** (so the named-instrument filter keeps the whole group).
3. **Distinctive-token overlap**, typo-tolerant: query tokens fuzzily matched (`SequenceMatcher ratio  $\geq 0.78$` , `len  $\geq 4$` — lets `qanuan/qanoon` hit `qanun`, `:324`) against title tokens. Accept if: query covers the **entire** title token set; OR ≥ 2 title words and $\geq 60\%$ coverage; OR it hits a token **unique to one document corpus-wide** and not in `_GENERIC_TITLE_WORDS` (so `shahadat` or `laundering` alone is decisive, but `procedure` or `evidence` is not). Tie-break ranking prefers more covered words $\rightarrow$ higher coverage $\rightarrow$ base act.
4. **Acronym-of-query**: initials of 2–5 consecutive query words matched against short doc identifier tokens (`anti narcotics force` $\rightarrow$ `anf`).

`related_document_ids(title)` (`:497`): same-group siblings (volumes) plus amendment $\leftrightarrow$ base links — used for the group soft-boost in retrieval.

12. Embeddings

`Embedder` (`embeddings.py:16`) lazily instantiates a fastembed `TextEmbedding` (CPU-only ONNX providers, `max_length=512` tokens).

- `embed_documents` (`:37`): batched `passage_embed`; on any exception the batch size halves and retries (OOM resilience), raising only at `batch_size=1`.
- `embed_query` (`:59`): single `query_embed` (bge models embed queries and passages differently — fastembed applies the right instruction prefixes).
- `dimension` is read from the model (768 for bge-base).

Note: chunks longer than 512 tokens are silently truncated by the model — the 2000-char chunk cap keeps most chunks under that, and the `SECTION:` prefix at the start guarantees the section identity is always inside the embedded window.

13. Vector Storage (Qdrant)

`QdrantChunkStore` (`storage.py:14`) wraps a **local-mode** client (`QdrantClient(path="qdrant_data")`) — embedded, no server).

- `reset (:19)`: **delete-then-create**, not `recreate_collection`. In local mode `recreate_collection` does NOT reliably clear data: a smaller rebuild overwrites only the low point-IDs and leaves the previous build's higher-index points orphaned, mixing stale chunks into the "fresh" index (this caused duplicate/stale results until fixed).
- `ensure_collection_exists (:34)`: for incremental upserts.
- `upsert (:45)`: point ID = `uuid5(NAMESPACE_URL, chunk_id)` — deterministic, so re-indexing the same file overwrites instead of duplicating. The **entire chunk dataclass is the payload** (text + metadata included).
- `scroll_all (:60)`: paginated full scan (256/page) — used to rebuild BM25/catalog and for incremental-indexing detection.
- `search (:79)`: `query_points` cosine KNN with payloads.

Vector distance: cosine. The store is the single source of truth; BM25 and the catalog are derived in memory from it at startup.

14. Hybrid Retrieval

`HybridRetriever` (`retrieval.py:44`). At construction (`_refresh_indexes, :65`) it scrolls all chunks into memory, builds `BM25Okapi` over `tokenize_for_search(chunk.text)`, and builds the `DocumentCatalog`.

14.1 Channels

- **Dense** (`_dense_search, :91`): embed query → Qdrant KNN.
- **Lexical** (`_lexical_search, :101`): BM25 scores over all chunks, top-k with score > 0.

14.2 Fusion — weighted Reciprocal Rank Fusion (`_fuse, :109`)

For each channel, a chunk at rank r contributes $\text{weight} / (\text{rrf_k} + r)$:

$$\text{fused} = 0.45 / (60 + \text{rank_dense}) + 0.55 / (60 + \text{rank_bm25})$$

Score magnitudes from the two channels never mix — only ranks — so BM25's unbounded scores can't drown cosine similarities. The fused score is subsequently **multiplied** by every boost in §16, and sentinel scores (5000/500) from the fallback scan deliberately dwarf it.

14.3 **search(query) orchestration (:255)** — full flow

1. Extract **query_section** (§15.1) and **query_document** (§15.2).
2. **Terse-query gate**: document targeting is honoured only if the query is ≤ 90 chars (**_TERSE_QUERY_CHAR_MAX**) **or** pins an explicit section number. In a long scenario, act names ("ANF", "Punjab police rules") are usually incidental — hard-filtering to them would drop the other statutes the scenario needs.
3. **Section variant resolution** (**_resolve_section_variant**, :557): if the exact cited form isn't stored, try **section_number_variants** ("9c" $\rightarrow$ 9C $\rightarrow$ 9(c) $\rightarrow$ 9), scoped to the named act. Exact form first, so a genuinely distinct 20B never collapses to 20.
4. **Query expansion for dense/lexical** (**_expand_query_for_dense_section**, :247): appends **section N / article N / N**. anchor lines so embeddings/BM25 can hit headings phrased differently from the user.
5. **Pool widening**: with section+document (non-comparison) the candidate pools grow to 48/48/36 (dense/lex/fused); section without document $\rightarrow$ 36/36. Deep pools make sure the actual provision chunk is *somewhere* in the pool before filtering.
6. Dense + lexical $\rightarrow$ fuse.
7. **Named-instrument filter** (§16.1) when a document is targeted (skipped for comparison intent — comparisons need both acts).
8. **Document boosts** (§16.2).
9. **Section filtering & boosts** (§16.3–16.6), then document-query affinity.
10. **Multi-document coverage** (§16.7) — only when no specific section is targeted or the intent is comparison.
11. Return top **rerank_top_k** (10).

15. Query Understanding

15.1 Section number extraction — `_extract_section_from_query` (`retrieval.py:570`)

Only **two safe sources** are trusted (free prose is full of incidental numbers — "register no 19", "46 days", "12 kg", years like "1975"):

0. **Typo repair first:** `411 -A / 411- A / 411 - A` → `411-A` (suffix ≤3 letters so `1979 - Article` is never glued).
1. **Explicit keyword reference:** `section|article|rule|sec <num>, u/s <num>, §<num>`, tolerating filler (`section no. 9, section number 9, typo section o 14(1)(a)`). The **last** match wins so pasted statute text above the real question can't steal the number. Rejected when a **unit word** follows (`section for 46 days` — days/kg/grams/rupees/packets/years/...), when the number is a **statute year** (4 digits in 1500–2099, `_looks_like_year, :566`), or when it's a bare `1/2` (too noisy).
2. **Pure-reference query:** strip polite filler (`what is, tell me about, explain...`) — if the remainder is *just* a section token (`20B, 337-I.`), that's the target.
3. **Terse-only:** leading `<num> of <doc>` form (`339-A of crpc`) — only for queries ≤90 chars so "3 of the accused were..." never matches.

All extracted numbers pass through `normalize_section_number` (§21).

15.2 Document context — `_extract_document_context (:366)`

Delegates to `DocumentCatalog.match_query` (§11.3). Returns the catalog's display title ("canonical" name), used by filters/boosts below.

16. Ranking: Filters & Boosts

All boosts multiply `fused_score`, then re-sort. In application order:

16.1 Named-instrument filter

(`_filter_candidates_by_named_instrument`, :174)

When an act is targeted (non-comparison):

- Keep only candidates whose path/title token-matches the canonical name (`_document_name_matches_canonical`, :144: $\geq \frac{2}{3}$ of the canonical's significant words must appear in the chunk's title/alias/filename blob — hyphen/space tolerant).
- **Authoritative section scan:** even within the filtered set, the true provision chunk may sit at a near-zero fusion score (a TOC stub or a section merely *mentioning* "154" can outscore it). `_fallback_chunks_for_named_act_section` (:154) scans the **entire corpus** for chunks of that act whose `section_number` metadata equals the target (sentinel score 5000) or whose text pattern-matches it (500), then upgrades/injects them into the pool.
- If the filter empties the pool entirely, the scan result alone is used; if even that is empty, the unrestricted pool is kept (graceful degradation).

16.2 Document boosts

- Target act chunks: **×2.0** (`_boost_document_matches`, :394).
- Same document-group siblings — other volumes, amendment acts: **×1.35** (`_boost_related_group_documents`, :375), via `catalog.related_document_ids`.

16.3 Section keyword filter

(`_filter_by_section_keyword`, :652)

Partitions candidates into exact-matches vs rest; exact matches (sorted by score) are moved wholesale ahead of everything else. A chunk "exactly matches" if any of:

- text pattern match (`chunk_text_matches_section_number`, §21);
- `section_number` metadata == target (normalized);
- a `section_path` heading parses to the target number.

16.4 Section boost (`_boost_section_matches`, :708)

- `amendment_target_sections` metadata contains target: **×3.0**;
- `section_number` == target: **×3.0**;
- amendment-section list contains target: **×2.5**;
- **Body-over-stub tiebreak**: among same-section matches, chunks that look like real provision text (≥250 chars or containing `(1)/shall/may/means`) get **×1.4** — so the body outranks a contents-page heading stub (otherwise the model answered "not found" while citing the stub).

16.5 Parent-child boost (`_boost_parent_child_matches`, :481)

- Querying a parent → its subsections (whose `parent_section/section_hierarchy` contains the target): **×1.8**.
- Querying a subsection → chunks whose `section_number` startswith the target: **×1.5**.

16.6 Amendment-aware boost (`_boost_amendment_matches`, :513)

Chunks with detected amendments: **×1.6**; chunks whose amendment references cite the queried section: **×1.4**. (Combined with the prompt's AMENDMENT NOTICE, §17.4.)

Then **document-query affinity** (`_apply_document_query_affinity`, :232): when the same section number exists in several laws, chunks whose title/filename tokens overlap the query get up to **×1.55** — a soft version of document targeting that works even when the catalog matched nothing.

16.7 Multi-document coverage (`_ensure_multi_document_coverage`, :421)

For open/comparison queries only. Two passes:

1. Reserve a slot for each document's best chunk — **but only if** that chunk scores $\geq$ 45% of the overall top score (`coverage_floor`). Without the floor, weak off-topic documents pushed out the correct document's strong chunks and the model cited the wrong file.
2. Fill remaining slots with the highest-scored unused chunks. Re-sort, cap at 10.

17. Intent Classification & Prompt Engineering

All in `rag_pipeline/prompts.py`.

17.1 `classify_query_intent` (`prompts.py:27`)

Regex classifier over the lowercased question, priority order (first match wins):

Priority	Intent	Trigger summary
0 (fast-path)	<code>simple_lookup</code>	interrogative + <code>section/article/rule <num></code> in the last 1200 chars — wins even when a pasted statute above contains "punishment" etc.
1	<code>comparison</code>	compare/difference/vs/versus/between... and/distinguish/contrast/similar/both...and
2	<code>penalty</code>	punish/penalty/sentence/imprisonment/fine/consequences
3	<code>procedural</code>	procedure/process/steps to/how to/filing a case/requirements/conditions
4	<code>cross_reference</code>	related sections/cross-reference/which sections apply/applicable sections
5	<code>explanation</code>	explain/meaning of/interpret/define/scope of/when does... apply/purpose of
default	<code>simple_lookup</code>	

The intent drives THREE things: prompt template (here), reasoning effort (`api.py:51`), and retrieval breadth (comparison unlocks multi-doc; `retrieval.py:294,318,354`).

17.2 System prompt (**SYSTEM_PROMPT**, **prompts.py:134**)

Persona (Pakistani-law analyst) + citation format + **CRITICAL RULES**, each added after a real failure:

- never fabricate provisions; state insufficiency explicitly;
- **NO-RELABEL rule**: never present another section's text under the requested number (the model once "found" a section by relabeling a different one when a doc-name typo broke matching);
- abbreviation equivalences (CNS Act/CNSA, CrPC, PPC, QSO, AMLA, ANF Act) and typo tolerance — never refuse over a short form;
- subsection semantics: 9(c)/9-C is satisfied by parent section 9's text; lettered sections (20B) stay distinct;
- **COMPLETE QUOTES**: enumerate every clause, never truncate with **.../etc.** (fixes the mid-provision truncation bug — it was a prompt conciseness rule, not a token cap);
- **AMENDMENTS**: answer with the current amended text, then note what changed.

17.3 Intent templates (**prompts.py:175–416**)

Six intent templates + two legacy (**COMPREHENSIVE_LEGAL_PROMPT**, **RECOMMENDATIONS_PROMPT**). All take {question}, {context}, {cross_references}.

- **SIMPLE_LOOKUP (:175)** — the most engineered. Prescribes exact markdown output (**## heading** → plain-language sentences → blockquoted provision, one clause per > line → ****Citation:**** → optional ****Significance:****), plus a "VERIFY BEFORE YOU ANSWER" battery: excerpt identity = its (**Section Number:**) value, never an internal cross-reference; trust matching excerpts even with garbled parents; parent-section answers for clause requests; refusal text when truly absent. **FORMATTING RULES** define "verbatim = same words, not same line-wrapping" — rejoin PDF-wrapped lines, break only before enumerated clauses (pairs with **remark-breaks** rendering in the UI).
- **EXPLANATION (:246)** — answer → provision → interpretation → scope → related provisions → takeaway.
- **COMPARISON (:267)** — overview → markdown comparison table → key differences → similarities → practical implications.

- **PROCEDURAL** (:295) — step-by-step with a per-step citation, requirements, timelines, authorities.
- **PENALTY** (:321) — offense/classification/punishment (imprisonment, fine, both)/aggravating/mitigating/additional consequences; exact penalty text, no paraphrasing of amounts.
- **CROSS_REFERENCE** (:346) — provisions grouped by document, interconnections, gaps/overlaps.
- **RECOMMENDATIONS** (:401) — used by the (currently dormant) recommendations path in `api.py:242`.

17.4 Context assembly — `build_legal_prompt` (`prompts.py:545`)

1. Auto-classify intent if not passed; select template (intent beats legacy `prompt_type` except `recommendations`).
2. `detect_cross_references` (:487): when chunks span >1 document, emit a CROSS-DOCUMENT block listing each doc + its section numbers and a same-number- in-multiple-laws disambiguation instruction.
3. `detect_amendment_relationship` (:449): groups chunks by `document_group_id`; when a group contains ≥2 distinct titles including an amendment, emits the AMENDMENT NOTICE (answer from the amended text; a single self-consolidated "amended upto..." doc does not trigger it).
4. `format_context_with_sources` (:513): renders chunks grouped by document with ==== document banners and per-excerpt headers `[EXCERPT i] [<section label>]` (Section Number: N) — these exact markers are what the SIMPLE_LOOKUP verification rules refer to.

18. LLM Layer & Adaptive Reasoning

(Defined in `api.py`; full mechanism in `REASONING_MECHANISM.md`.)

- **Model:** `OPENAI_MODEL` env or default `gpt-5-mini` (`api.py:35`).
- `_is_reasoning_model` (:47): `gpt-5*`, `o1`, `o3`, `o4`.
- `_reasoning_effort_for` (:51): intent + length → effort:
 - `simple_lookup` ≤200 chars → `minimal` (no thinking; fast/cheap)
 - `explanation/penalty` ≤300 chars → `low`

- everything else (comparison, procedural, cross-reference, any long/scenario question) → **medium**
- o-series doesn't accept **minimal** → bumped to **low**.
- `_completion_params (:67)`: reasoning models get `{max_completion_tokens: answer+4000 headroom, reasoning_effort}`; classic models get `{temperature: 0.3, max_tokens}` (reasoning models reject those keys).
- Scenario questions have no dedicated intent — they're caught by the length fallthrough and/or the penalty/procedural/cross-ref phrasing of their tail, and by the retriever's terse-query gate (§14.3 step 2) which disables single-document lock-in for them.

Fallback chain (`stream_answer`, `api.py:313`): no API key → extractive `generate_fallback_answer (:468`, first relevant context lines) + section-based recommendations; LLM exception mid-stream → same fallback path. Either way the stream always terminates with `sections` (if any) and `done`.

19. API Server & SSE Streaming Protocol

FastAPI app (`api.py:83`), CORS fully open, pipeline built once at startup (`@app.on_event("startup")`, `:99`).

Endpoints

Route	Purpose
<code>GET /health</code>	liveness probe
<code>POST /chat</code>	main endpoint — SSE stream (below)
<code>POST /query</code> , <code>GET /query?q=</code>	debug: returns assembled prompt + full chunk list with all four scores
<code>GET /debug/sections?doc=&section=&contains=</code>	inspect stored section numbers per document

`/chat` flow (`api.py:140`)

1. Classify intent → SSE thinking: "Understanding the question (<label>)..."
2. Surface resolved targets → thinking: "Identified law: ... • section: ...".
3. thinking: "Searching the indexed legal documents..." → `retriever.search()`. Empty → a polite "couldn't find" answer event and return.
4. Build context from top 10 candidates: per-chunk block with source/file/section path/section number and `reflow_provision_text-ed` text (§21); track distinct docs → thinking: "Retrieved N excerpt(s) from M document(s)".
5. `build_legal_prompt(question, context, retrieved_chunks=top10, query_intent=...)`.
6. `stream_answer`: effort-dependent thinking line → OpenAI streaming call → answer events per token delta → sections event (`build_section_references_from_chunks`, §21) → done.
7. Any exception → error event.

SSE event schema (each line data: <json>\n\n, _sse, api.py:78)

type	content	Meaning
thinking	string	progress line for the UI thinking trail
answer	string	answer token delta
sections	[{type, number, law, full_reference, relevance}]	extracted citations
recommendations	string	fallback-mode only
done	—	terminator
error	string	fatal error

Sources are deliberately **not** sent to the UI; they're printed to the server terminal only (`api.py:367`).

20. Frontend

React + Vite SPA in `frontend/`, talking to `http://localhost:8000` (override: `VITE_API_URL`).

App.jsx

Holds all state in memory (no persistence): `conversations[]` (`id = Date.now()`, title, messages, createdAt), current conversation id, and light/dark theme persisted to `localStorage` and applied via `data-theme` attribute. `addMessageToConversation` upserts by message id — this is what makes streaming work: the assistant message is re-sent with the same id and growing content.

utils/api.js — SSE client

`queryAPI(question, onChunk)` uses raw `fetch` + `ReadableStream` reader (axios can't stream): decodes chunks, splits on newlines, parses every `data:` line, and maps server events to callbacks (`answer` → `chunk` for legacy naming; `sources` events are swallowed). Invalid JSON lines are skipped. A separate axios client exists only for `/health`.

ChatArea.jsx

- Submits on Enter (Shift+Enter = newline); input cleared immediately; user message appended; conversation title auto-set from the first message (30-char preview).
- Creates one assistant message with `isStreaming: true`, then `pushUpdate()` re-emits it on every event with accumulated `fullAnswer` and `thinkingLines`.
- `thinking` events append to the trail; `isThinking` stays true until the first answer token arrives. `done` flips `isStreaming` off.
- Auto-scroll on every update; typing-indicator bubble while waiting; error messages rendered as a flagged assistant bubble.

MessageBubble.jsx

- **ThinkingBlock:** while reasoning (no answer text yet) shows a live spinner + step list (last step highlighted as `current`); once the answer streams, the trail collapses into a `<details>` "Thought process · N steps" summary.

- Answer rendered with `react-markdown` + `remark-breaks` (every newline = `<br>` — this is why the backend reflows PDF line-wraps and the prompt forbids mid-sentence breaks), with custom renderers for headings, blockquotes (provision quotes), lists, code, and links (styled in `MessageBubble.css`).

Sidebar.jsx

Conversation list with client-side title search, relative date labels (Today/Yesterday/weekday/date), hover-to-delete, theme toggle.

21. Shared Utilities (`rag_pipeline/utils.py`)

- `normalize_section_number (:17)` — canonical form so spellings compare equal: separator (space/hyphen/underscore/none) and case are irrelevant. Single-letter suffix joins (`35-a` → `35A`); multi-letter hyphenates (`337 iii` → `337-III`); plain/dotted/parenthesized forms unchanged.
 - `section_number_variants (:39)` — progressively broader equivalents, most specific first: `9C` → `[9C, 9(c), 9]`, `14(1)(a)` → `[14(1)(a), 14(1), 14]`. Used to remap clause citations to the parent section the index actually stores.
 - `chunk_text_matches_section_number (:88)` — boundary-safe regex battery for "does this text contain section N as a heading/reference": `Section N`, `§N`, line-leading `N./N)/N:` forms (with a ≤6-char garbage tolerance at line start), and `SECTION: <label containing N>`.
 - `reflow_provision_text (:118)` — rejoins PDF-wrapped lines inside each paragraph; a new line is kept **only** when it starts an enumerated clause (`((a), (i), (1), 2A))` or `Explanation/Provided/Illustration`. Fixes shattered quotes at the source (pairs with `remark-breaks` in the UI).
 - `clean_markdown_formatting (:143)` — strips `##*/*` (fallback recommendations path).
 - `build_section_references_from_chunks (:152)` — converts retrieved chunks into the UI `sections` payload: normalized number + doc title, deduped, with a relevance score derived from `fused_score` (clamped 0.5–0.99).
 - `chunk_metadata (:10)`, `normalize_doc_blob (:83)` — small accessors.
-

22. Operations

```
# 1. Install
pip install -r requirements.txt

# 2. Index (incremental by default; only new/changed files)
python ingest.py --data-dir data

# Full clean rebuild (required after chunking/heading changes)
python ingest.py --data-dir data --force-reindex

# 3. API
uvicorn api:app --reload --port 8000

# 4. Frontend
cd frontend; npm install; npm run dev    # Vite, default http://localhost:5173
```

- `.env` needs `OPENAI_API_KEY` (and optionally `OPENAI_MODEL`). Without a key the system still runs in extractive-fallback mode.
- First run downloads the bge ONNX model into the fastembed cache.
- Debugging retrieval: `GET /query?q=...` shows the exact prompt + per-chunk dense/lexical/fused scores; `GET /debug/sections?doc=penal§ion=302` inspects stored metadata; retrieval logs every stage (extraction, widening, each boost) via `logging` (`retrieval.py:272` onward).
- Reindex history lives in the `reindex_*.log` files in the root.

23. Known Gotchas & Fixed Bugs

Documented because each shaped a guard you'll find in the code:

Bug	Root cause	Fix
0% section metadata coverage	<code>fix_pdf_spacing</code> used <code>str.split()</code> over whole text → all newlines destroyed → heading detection saw one giant line	line-by-line repair (<code>text_normalization.py:183</code>) + reindex

Bug	Root cause	Fix
Answers cut off mid-provision	prompt "be concise" rule, not a token cap	COMPLETE QUOTES rule (prompts.py:169)
Model fabricated a section by relabeling another	doc-name typo broke matching; model "helpfully" renamed an excerpt	NO-RELABEL rules (prompts.py:149 , SIMPLE_LOOKUP verify block) + fuzzy doc matching (document_catalog.py:324)
Phantom sections from page footers / TOC / cross-refs	`3	Page, dotted-leader TOC lines, wrapped Section 30 , may pass...` lines parsed as headings
Shall--- became a level-1 heading, erased CNS s.8	dash-terminated Title-Case wrapped line	trailing-punctuation reject in _looks_like_title (hierarchy.py:82)
Stale/duplicate chunks after rebuild	qdrant-local recreate_collection leaves orphan points	delete-then-create in reset() (storage.py:19)
"Not found" though the section existed	TOC stub outranked provision body	body-over-stub ×1.4 (retrieval.py:786) + authoritative scan (retrieval.py:154)
Wrong act cited on open questions	coverage pass reserved slots for weak documents	45% coverage floor (retrieval.py:449)
337-A lost its number	PDF split 337- / A. across lines	dangling-suffix merge (chunking.py:46)
"cns act" / "crpc" / "9c" unresolved	abbreviations/clauses not in index vocabulary	catalog acronyms + _KNOWN_ABBREVIATIONS + section_number_variants

24. Limitations & Future Work

- **Reranker hook unused** — `rerank_score` exists on `RetrievedChunk` and `reranker_model` in config, but no cross-encoder is wired in.
- **In-memory everything at query time** — all chunks + BM25 live in RAM (fine at 10k chunks; revisit at 100k+). Catalog/BM25 rebuild requires process restart after external index changes.
- **No conversation memory** — each question is answered statelessly; the frontend keeps history for display only, and it's lost on refresh (no backend persistence).
- **Incremental indexing is path-based only** — a *modified* file with the same path is not re-indexed unless `--force-reindex` (no content hashing).
- **512-token embedding window** — very long chunks are truncated in embedding space (BM25 still sees the full text).
- **Single LLM provider** — OpenAI only; Azure env vars exist in `.env.example` but are not read.
- **Security** — CORS is `*`; debug endpoints are unauthenticated; intended for local use only.